Machine Learning Algorithm Comparison Report

CMPE475 Machine Learning

Prepared by

KUANDYK KYRYKBAYEV

Instructor

Doç. Dr. İsmail Koç

Kadir Has University

Department of Computer Engineering

Spring 2026

Submission Date
[15th April 2026]

Contents

1. INTRODUCTION 4
2. DATABASE SELECTION 5
2.1 Dataset Description 5
2.2 Why This Dataset? 5
2.3 Class Distribution 5
3. TARGET SELECTION 6
3.1 Classification vs Regression 6
3.2 Target Variable 6
4. FEATURE ENGINEERING 7
4.1 release_year (Direct Use) 7
4.2 duration_num (Extracted from duration) 7
4.3 rating_encoded (Label-Encoded) 7
4.4 country_US (Binary) 7
4.5 country_India (Binary) 7
4.6 num_cast (Count) 7
4.7 has_director (Binary) 7
4.8 num_genres (Count) 7
4.9 genre_drama (Binary) 7
4.10 genre_comedy (Binary) 8
4.11 genre_international (Binary) 8
4.12 month_added (Extracted from date_added) 8
4.13 description_len (Count) 8
5. DATA PREPROCESSING 9
5.1 Missing Value Handling 9
5.2 Train/Test Split 9
5.3 Feature Scaling 9
6. EVALUATION METRICS 10
6.1 Accuracy 10
6.2 Precision 10
6.3 Recall (Sensitivity) 10
6.4 F1-Score 10
6.5 ROC-AUC (Area Under the ROC Curve) 10
7. ALGORITHMS 11
7.1 Bayesian (Naive Bayes) - GaussianNB 11
7.2 Decision Trees - DecisionTreeClassifier(max_depth=5) 11
7.3 Random Forest - RandomForestClassifier(n_estimators=100) 11
7.4 Simple Linear Regression - LinearRegression() with 1 feature 11
7.5 Multiple Linear Regression - LinearRegression() with all features 11
7.6 Logistic Regression - LogisticRegression(max_iter=1000) 11

7.7 KNN (K-Nearest Neighbors) - KNeighborsClassifier(n_neighbors=5) 11
7.8 SVM (Support Vector Machine) - SVC(kernel='rbf', probability=True) 12
7.9 LSTM (Long Short-Term Memory) – Homework 2 12
7.10 GRU (Gated Recurrent Unit) – Homework 2 12
8. RESULTS 14
8.1 Result Table (10 models: 8 traditional ML + 2 ANN) 14
8.2 Analysis – Traditional ML Models 14
8.3 Analysis – ANN Models (Homework 2 Findings) 14
9. PLOTS 16
9.1 Confusion Matrices (confusion_matrices.png) 16
9.2 ROC Curves (roc_curves.png) 16
9.3 Metrics Bar Chart (metrics_comparison.png) 17
9.4 ANN Training History (ann_training_history.png) 17
10. CONCLUSION 18
11. FILES IN THIS PROJECT 19
12. LIBRARIES AND TOOLS 19
13. REFERENCES 19

1. INTRODUCTION

This report presents a comparative analysis of ten machine learning algorithms applied to a TV shows and movies dataset. The objective is to classify content as either a "Movie" or a "TV Show" using extracted features from the dataset, evaluate each algorithm using five classification metrics, and visualize the results through plots.

In Homework 2, two Artificial Neural Network (ANN) models – LSTM and GRU - were added to extend the original eight traditional ML algorithms.

The dataset used is tv-shows.csv, a collection of 9,338 titles from streaming platforms including Netflix. The dataset is in tabular CSV format and falls under the Tabular modality category.

Git repo name is still the same: CMPE475-ASSIGNMENT-1-KUANDYK-KYRYKBAYEV

20211701013

https://github.com/kuandyque/CMPE475-ASSIGNMENT-1-KUANDYK-KYRYKBAYEV.git

The dataset used is tv-shows.csv, a collection of 9,338 titles from streaming platforms including Netflix. The dataset is in tabular CSV format and falls under the Tabular modality category.

2. DATABASE SELECTION

2.1 Dataset Description

The dataset contains 9,338 rows and 13 columns. Each row represents a single media title (movie or TV show) with the following attributes:

- id: Unique identifier for each title

- type: Content type - "Movie" or "TV Show" (this is our classification target)

- title: Name of the content

- director: Director(s) of the content (30.6% missing)

- cast: Actors in the content (10.0% missing)

- country: Country of production (10.3% missing)

- date_added: Date the title was added to the platform

- release_year: Year the content was originally released

- rating: Content rating (PG-13, TV-MA, R, etc.)

- duration: Duration in minutes (for movies) or number of seasons (for TV shows)

- listed_in: Genre categories

- description: Brief description of the content

- platform: Streaming platform name

2.2 Why This Dataset?

This dataset was chosen because:

(a) It is in tabular CSV format, which is compatible with all eight required algorithms.

(b) It contains a natural binary target variable (Movie vs TV Show).

(c) It has sufficient size (9,338 rows) for meaningful training and evaluation.

(d) It contains diverse feature types (numeric, categorical, text) that require practical feature engineering.

(e) It contains real-world imperfections like missing values, requiring data cleaning.

2.3 Class Distribution

The target variable (type) has the following distribution:

- Movie: 6,528 samples (69.9%)

- TV Show: 2,810 samples (30.1%)

This moderate class imbalance (approximately 2.3:1) justifies the use of multiple evaluation metrics beyond simple accuracy.

3. TARGET SELECTION

3.1 Classification vs Regression

Classification was chosen over regression for the following reasons:

(a) The target variable (type) is categorical with exactly two classes.

(b) Six of the eight required algorithms (Naive Bayes, Decision Trees, Random Forest, Logistic Regression, KNN, SVM) are primarily designed for classification.

(c) Classification metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC) provide more nuanced performance analysis than regression metrics.

(d) Classification results are more intuitive to interpret and visualize (confusion matrices, ROC curves).

3.2 Target Variable

The type column serves as the target variable:

- Class 0: Movie

- Class 1: TV Show

The model's task is to predict whether a given title is a Movie or a TV Show based on its metadata features.

4. FEATURE ENGINEERING

Machine learning algorithms require numeric input. The raw dataset contains text, dates, and categorical values that must be transformed into meaningful numeric features. Thirteen features were engineered:

4.1 release_year (Direct Use)

The release year is already numeric and used directly. It captures temporal trends in content production.

4.2 duration_num (Extracted from duration)

The numeric value is extracted from the duration string using regular expression. For example, "128 min" becomes 128 and "3 Seasons" becomes 3. This is the most discriminative feature because movies have values in the range 60-300 (minutes) while TV shows have values in the range 1-15 (seasons).

4.3 rating_encoded (Label-Encoded)

Content ratings (TV-MA, PG-13, R, TV-Y, etc.) are converted to integers using scikit-learn's LabelEncoder. Movies tend to use MPAA ratings while TV shows use TV-specific ratings.

4.4 country_US (Binary)

A binary flag indicating whether "United States" appears in the country field. The US has different production ratios of movies to TV shows compared to other countries.

4.5 country_India (Binary)

A binary flag indicating whether "India" appears in the country field. India produces a disproportionately high number of movies (Bollywood).

4.6 num_cast (Count)

The number of cast members listed, calculated by counting commas in the cast string and adding 1. TV shows often list larger ensemble casts.

4.7 has_director (Binary)

A binary flag indicating whether a director is listed. TV shows are far more likely to have no director listed (since they have multiple directors across seasons). Of the 2,855 missing director values, a disproportionate number belong to TV shows.

4.8 num_genres (Count)

The number of genre tags listed in the listed_in field. TV shows tend to be tagged with more categories.

4.9 genre_drama (Binary)

A binary flag for drama-related content, detected by searching for "Drama" in the listed_in field.

4.10 genre_comedy (Binary)

A binary flag for comedy-related content, detected by searching for "Comed" in the listed_in field.

4.11 genre_international (Binary)

A binary flag for international content, detected by searching for "International" in the listed_in field.

4.12 month_added (Extracted from date_added)

The month component of the date_added field (1-12), extracted using pandas datetime parsing. Captures seasonal content release patterns.

4.13 description_len (Count)

The character count of the description text. Different content types may have different description length conventions.

5. DATA PREPROCESSING

5.1 Missing Value Handling

Missing values were handled as follows:

- Numeric features: Filled with 0 using fillna(0)

- Text features: Filled with empty string before feature extraction

- Target variable: No missing values (0 missing)

5.2 Train/Test Split

The dataset was split 80/20:

- Training set: 7,470 samples (80%)

- Test set: 1,868 samples (20%)

Stratified sampling (stratify=y) was used to maintain the same 70/30 class ratio in both sets. A fixed random seed (random_state=42) ensures reproducibility.

5.3 Feature Scaling

StandardScaler was applied to normalize all features to zero mean and unit variance. This is critical for distance-based algorithms (KNN, SVM) where features with larger ranges would dominate distance calculations. The scaler was fit on training data only and then applied to both training and test data to prevent data leakage.

6. EVALUATION METRICS

Five classification metrics were used to evaluate each algorithm:

6.1 Accuracy

Formula: (TP + TN) / Total

Measures the overall percentage of correct predictions. Simple and intuitive but can be misleading on imbalanced datasets.

6.2 Precision

Formula: TP / (TP + FP)

Measures the proportion of positive predictions that are correct. High precision means few false positives (false alarms).

6.3 Recall (Sensitivity)

Formula: TP / (TP + FN)

Measures the proportion of actual positives that are correctly identified. High recall means few missed cases.

6.4 F1-Score

Formula: 2 * (Precision * Recall) / (Precision + Recall)

The harmonic mean of Precision and Recall. Provides a single score that balances both metrics. More informative than accuracy on imbalanced datasets.

6.5 ROC-AUC (Area Under the ROC Curve)

Measures the model's ability to distinguish between classes across all possible decision thresholds. AUC = 1.0 indicates perfect classification; AUC = 0.5 indicates random guessing.

7. ALGORITHMS

7.1 Bayesian (Naive Bayes) - GaussianNB

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It calculates the posterior probability of each class given the observed features, assuming feature independence. The Gaussian variant assumes features follow normal distributions. Despite the "naive" independence assumption, it performs well on many real-world tasks and is extremely fast to train.

7.2 Decision Trees - DecisionTreeClassifier(max_depth=5)

Decision Trees build a flowchart-like structure where each internal node tests a feature, each branch represents a decision, and each leaf assigns a class. The algorithm selects the feature and threshold at each node that best separates the classes using metrics like Gini impurity. The max_depth=5 parameter limits tree depth to prevent overfitting. Decision Trees are highly interpretable but can be unstable.

7.3 Random Forest - RandomForestClassifier(n_estimators=100)

Random Forest is an ensemble method that builds 100 independent Decision Trees, each trained on a random subset of the data and features. Predictions are made by majority voting across all trees. This approach reduces overfitting and variance compared to a single Decision Tree, producing more robust and accurate predictions.

7.4 Simple Linear Regression - LinearRegression() with 1 feature

Simple Linear Regression fits a straight line y = mx + b using a single feature (release_year). Since it outputs continuous values rather than class labels, a threshold of 0.5 is applied for classification. "Simple" means exactly one independent variable is used. This baseline demonstrates the limitation of using a single feature for classification.

7.5 Multiple Linear Regression - LinearRegression() with all features

Multiple Linear Regression extends Simple Linear Regression to use all 13 features: y = w1*x1 + w2*x2 + ... + w13*x13 + b. Each feature receives its own weight. The output is again thresholded at 0.5. This demonstrates how adding more features improves predictive power, even with a simple linear model.

7.6 Logistic Regression - LogisticRegression(max_iter=1000)

Despite its name, Logistic Regression is a classification algorithm. It applies a sigmoid function to the linear combination of features, producing a probability between 0 and 1. If the probability exceeds 0.5, the sample is classified as TV Show. Logistic Regression is the proper classification counterpart to Linear Regression. The max_iter=1000 parameter allows sufficient iterations for convergence.

7.7 KNN (K-Nearest Neighbors) - KNeighborsClassifier(n_neighbors=5)

KNN classifies a new sample by finding its 5 nearest neighbors in feature space (using Euclidean distance) and assigning the majority class among those neighbors. KNN is a "lazy learner" that stores all training data and performs computation at prediction time. It requires scaled features because distance calculations are sensitive to feature magnitudes.

7.8 SVM (Support Vector Machine) - SVC(kernel='rbf', probability=True)

SVM finds the optimal hyperplane that maximally separates the two classes. It focuses on "support vectors" (training points closest to the boundary). The RBF (Radial Basis Function) kernel allows SVM to handle non-linear boundaries by mapping data into a higher-dimensional space. The probability=True parameter enables probability estimates needed for ROC-AUC calculation.

7.9 LSTM (Long Short-Term Memory) – Homework 2

LSTM is a type of Recurrent Neural Network (RNN) designed to learn long-range dependencies in sequential data. Unlike standard RNNs that suffer from the vanishing gradient problem, LSTM uses a gating mechanism with three gates: the forget gate (decides what information to discard), the input gate (decides what new information to store), and the output gate (decides what to output). Each gate uses sigmoid activations to control the flow of information through a cell state.

Architecture used:

- Input shape: (13, 1) – each of the 13 features treated as one timestep
- Layer 1: LSTM(64 units, return_sequences=True)
- Layer 2: LSTM(32 units)
- Layer 3: Dense(16, activation='relu')
- Layer 4: Dropout(0.3) – regularization to prevent overfitting
- Output: Dense(1, activation='sigmoid') – binary classification
- Optimizer: Adam (learning_rate=0.001)
- Loss: binary_crossentropy
- Training: max 50 epochs with EarlyStopping (patience=5), 15% validation split

Since our dataset is tabular (not sequential), the 13 features were reshaped into a pseudo-sequence of shape (samples, 13, 1), treating each feature as one timestep. This is a standard approach for applying RNNs to tabular data for comparison purposes.

7.10 GRU (Gated Recurrent Unit) – Homework 2

GRU is a simplified variant of LSTM that uses only two gates instead of three: the reset gate (controls how much past information to forget) and the update gate (controls how much new information to incorporate). GRU merges the cell state and hidden state into a single hidden state, making it computationally more efficient with fewer parameters than LSTM while maintaining similar performance on many tasks.

Architecture used:

- Input shape: (13, 1) – same reshaping as LSTM
- Layer 1: GRU(64 units, return_sequences=True)

- Layer 2: GRU(32 units)
- Layer 3: Dense(16, activation='relu')
- Layer 4: Dropout(0.3)
- Output: Dense(1, activation='sigmoid')
- Optimizer: Adam (learning_rate=0.001)
- Loss: binary_crossentropy
- Training: max 50 epochs with EarlyStopping (patience=5), 15% validation split

8. RESULTS

8.1 Result Table (10 models: 8 traditional ML + 2 ANN)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Bayesian (Naive Bayes)	0.9920	0.9893	0.9840	0.9866	0.9989
Decision Trees	0.9968	0.9947	0.9947	0.9947	0.9968
Random Forest	0.9968	0.9894	1.0000	0.9947	0.9999
Simple Linear Regression	0.6991	0.0000	0.0000	0.0000	0.6570
Multiple Linear Regression	0.9625	0.9227	0.9555	0.9388	0.9972
Logistic Regression	0.9963	0.9894	0.9982	0.9938	0.9998
KNN	0.9764	0.9481	0.9751	0.9614	0.9905
SVM	0.9930	0.9791	0.9982	0.9885	0.9999
LSTM	0.9845	0.9734	0.9751	0.9742	0.9977
GRU	0.9234	0.8238	0.9484	0.8817	0.9725

8.2 Analysis – Traditional ML Models

Best performing algorithms:

- Decision Trees and Random Forest tied for highest accuracy (0.9968) and F1-Score (0.9947).

- Random Forest achieved perfect recall (1.0000), correctly identifying every TV Show in the test set.

- Random Forest, Logistic Regression, and SVM all achieved ROC-AUC above 0.9998.

Worst performing algorithm:

- Simple Linear Regression performed poorly (Accuracy = 0.6991) because using only release_year as a single feature provides insufficient information. It predicted "Movie" for all samples, resulting in zero Precision, Recall, and F1 for the TV Show class.

Key insight:

- The dramatic improvement from Simple to Multiple Linear Regression (0.6991 to 0.9625 accuracy) demonstrates that using multiple features provides significantly more predictive power.

Why are scores so high?

- The duration_num feature provides near-perfect class separation. Movies are measured in minutes (typically 60-300) while TV shows are measured in seasons (typically 1-15). Most algorithms can easily exploit this strong signal.

8.3 Analysis – ANN Models (Homework 2 Findings)

LSTM Performance:

- LSTM achieved Accuracy = 0.9845, F1-Score = 0.9742, ROC-AUC = 0.9977.

- Training stopped at epoch 22 out of 50 (early stopping), indicating the model converged relatively quickly.

- LSTM performs comparably to KNN (0.9764) and SVM (0.9930), placing it in the upper-middle tier of all 10 models.

GRU Performance:

- GRU achieved Accuracy = 0.9234, F1-Score = 0.8817, ROC-AUC = 0.9725.

- Training stopped at epoch 5 out of 50 (very early stopping), suggesting the model did not have enough epochs to learn complex patterns before validation loss plateaued.

- GRU underperformed compared to LSTM, likely because its simpler gating mechanism (2 gates vs 3) may converge to a less optimal solution with fewer training iterations.

Comparison – Traditional ML vs ANN:

- Traditional ML models (especially Random Forest, Decision Trees, and Logistic Regression) outperformed both ANN models on this particular dataset.

- This is expected for tabular data with well-engineered features. RNNs (LSTM/GRU) are designed for sequential data (time series, text, speech). When applied to tabular data with pseudo-sequence reshaping, they cannot fully exploit their sequential learning capabilities.

- The overhead of neural network training (more parameters, gradient-based optimization) does not provide an advantage when simpler models can already achieve 99%+ accuracy using straightforward feature relationships.

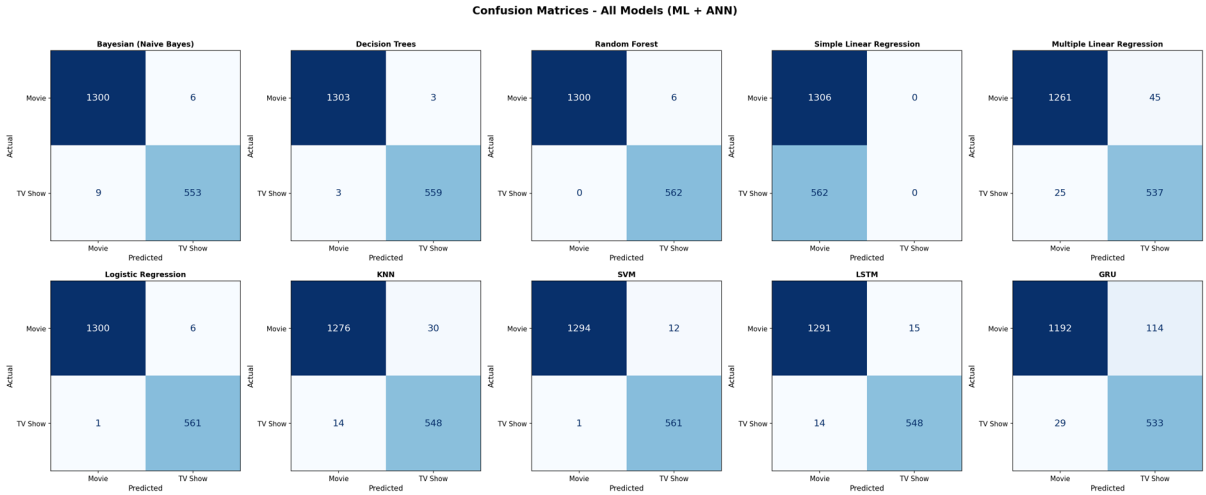
- LSTM outperformed GRU, likely due to its more expressive gating mechanism (3 gates vs 2) allowing finer control over information flow.

Key Takeaway:

- Algorithm selection should match the data modality. For tabular classification with well-separated features, traditional ML algorithms are sufficient and often superior. RNNs shine on truly sequential data where temporal dependencies matter.

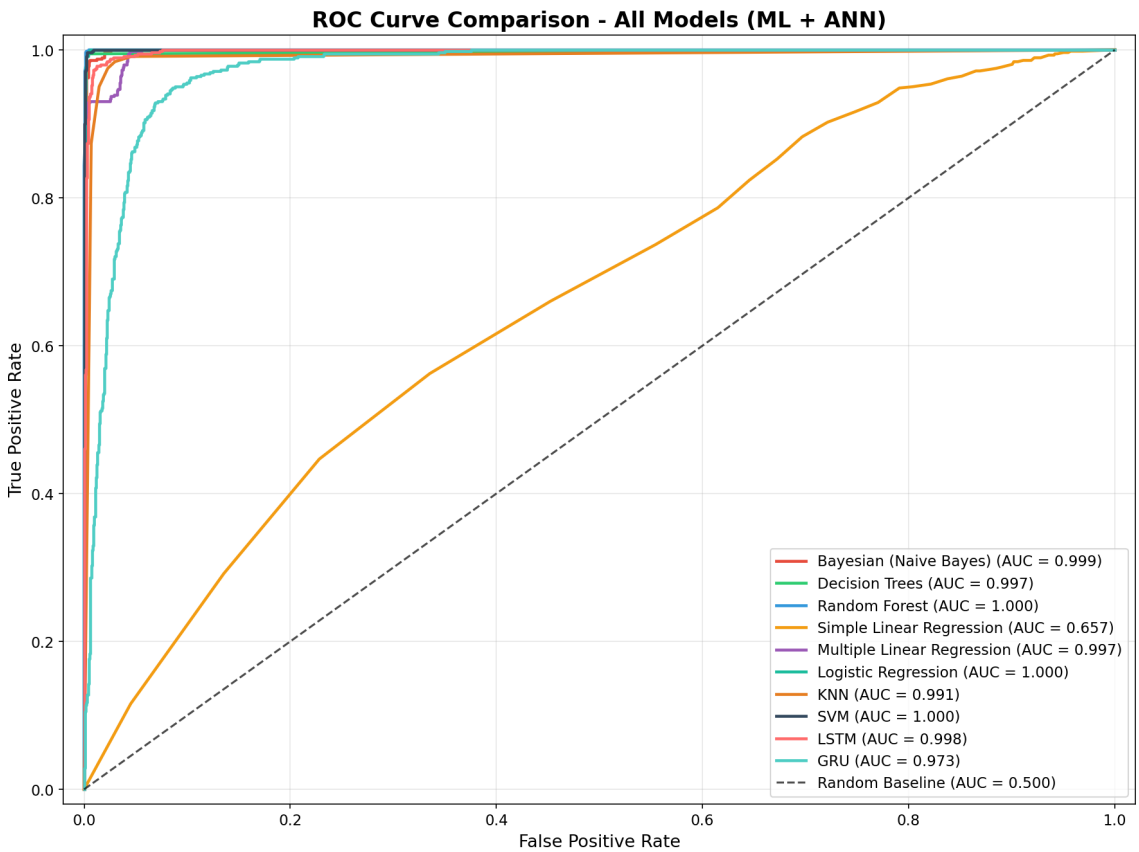
9. PLOTS

9.1 Confusion Matrices (confusion_matrices.png)



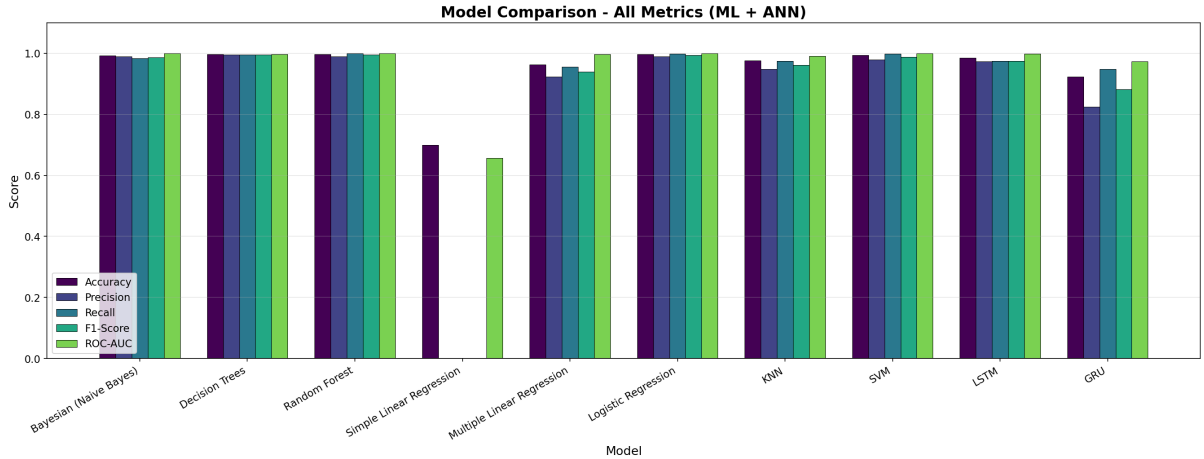
A 2x5 grid (10 panels) showing the confusion matrix for each model, including the new LSTM and GRU models. Each matrix is a 2x2 grid displaying True Negatives (Movie correctly predicted), False Positives (Movie incorrectly predicted as TV Show), False Negatives (TV Show incorrectly predicted as Movie), and True Positives (TV Show correctly predicted). The diagonal values represent correct predictions.

9.2 ROC Curves (roc_curves.png)



All ten ROC curves are overlaid on a single plot. The ROC curve plots True Positive Rate vs False Positive Rate at various decision thresholds. Models whose curves hug the top-left corner perform better. The dashed diagonal line represents random guessing (AUC = 0.5). Simple Linear Regression's curve is closest to the diagonal, while all other models including LSTM and GRU are near the top-left corner.

9.3 Metrics Bar Chart (metrics_comparison.png)



A grouped bar chart comparing all five metrics across all ten models. Each model has five bars (one per metric) allowing quick visual comparison. LSTM bars are comparable to KNN and SVM, while GRU bars are somewhat shorter, reflecting its lower performance on this tabular dataset.

9.4 ANN Training History (ann_training_history.png)

A 2x2 grid showing training and validation curves for both LSTM and GRU. The top row displays loss curves (training loss vs validation loss over epochs) and the bottom row displays accuracy curves. These plots help visualize how quickly each model converges and whether overfitting occurs. LSTM shows gradual convergence over 22 epochs, while GRU converges much faster (5 epochs) but to a lower performance level.

10. CONCLUSION

This assignment demonstrated the application of ten machine learning algorithms (8 traditional ML + 2 ANN) to a binary classification task using streaming platform data. Key findings include:

1. Random Forest and Decision Trees achieved the highest overall performance, with Random Forest achieving perfect recall.

2. Ensemble methods (Random Forest) outperformed individual algorithms in terms of stability and ROC-AUC.

3. Simple Linear Regression's poor performance highlights the importance of using multiple features.

4. Logistic Regression, as the proper classification counterpart to Linear Regression, achieved near-perfect results.

5. Feature engineering transforming raw text and categorical data into numeric features was essential for algorithm compatibility.

6. The duration_num feature was the most discriminative, providing near-perfect class separation.

7. Feature scaling was critical for distance-based algorithms (KNN, SVM) and neural networks (LSTM, GRU).

8. LSTM (Accuracy=0.9845) outperformed GRU (Accuracy=0.9234), demonstrating that the additional forget gate in LSTM provides finer control over information retention.

9. Both ANN models underperformed compared to the best traditional ML models on this tabular dataset, confirming that RNNs are best suited for truly sequential data rather than tabular feature sets.

10. Algorithm selection should match the data modality – for well-engineered tabular features, traditional ML algorithms are often sufficient and superior to deep learning approaches.

The complete code, results table, and all plots are available in the project repository.

11. FILES IN THIS PROJECT

https://github.com/kuandyque/CMPE475-ASSIGNMENT-1-KUANDYK-KYRYKBAYEV.git

- tv-shows.csv : Original dataset (9,338 rows, 13 columns)
- ml_assignment.py : Complete Python script with all 10 algorithms (8 ML + 2 ANN)
- results_table.csv : Output result table (10 models x 5 metrics)
- confusion_matrices.png : Confusion matrix plots for all 10 models (2x5 grid)
- roc_curves.png : Overlaid ROC curves for all 10 models
- metrics_comparison.png : Grouped bar chart of all metrics for 10 models
- ann_training_history.png : LSTM & GRU training/validation loss and accuracy curves
- EXPLANATION.md : Detailed technical explanation (Markdown)
- REPORT.txt : This report

12. LIBRARIES AND TOOLS

- Python 3.11
- pandas 2.2.1 : Data loading and manipulation
- numpy 1.26.4 : Numerical operations
- scikit-learn 1.8.0 : Machine learning algorithms and metrics
- matplotlib 3.10.8 : Plotting and visualization
- seaborn 0.13.2 : Statistical visualizations
- tensorflow 2.21.0 : Deep learning framework (LSTM, GRU)
- keras 3.14.1 : High-level neural network API (included with TensorFlow)

13. REFERENCES

1. Mendoza, M., & Chamorro, Á. (2025, February 2). *MARCOM003/TV-shows-netflix-disney · datasets at hugging face*. MarcoM003/TV-Shows-Netflix-Disney · Datasets at Hugging Face. https://huggingface.co/datasets/MarcoM003/TV-Shows-Netflix-Disney

2. Koç, İ. (2026). *Lectures: Machine Learning* [PowerPoint slides]. Kadir Has University